

SELLER AND INVENTORY MANAGEMENT

1.Create Seller Table

The Seller table stores the basic details of sellers, including their name, email, phone number, and address.

Attribute	Data Type	Constraint
Seller_ID	NUMBER	PRIMARY KEY
Seller_Name	VARCHAR2(50)	—
Email	VARCHAR2(100)	—
Phone	VARCHAR2(15)	—
Address	VARCHAR2(100)	—

```
SQL> CREATE TABLE Seller (  
2     Seller_ID NUMBER(5) PRIMARY KEY,  
3     Seller_Name VARCHAR2(50) NOT NULL,  
4     Email VARCHAR2(100) UNIQUE,  
5     Phone VARCHAR2(15),  
6     Address VARCHAR2(150)  
7 );  
  
Table created.
```

Insert Seller Table

```
Table created.  
  
SQL> INSERT INTO Seller VALUES  
2  (1, 'ABC Traders', 'abc@gmail.com', '9876543210', 'Chennai');  
  
1 row created.  
  
SQL> INSERT INTO Seller VALUES  
2  (2, 'Smart World', 'smartworld@gmail.com', '9876543211', 'Bangalore');  
  
1 row created.  
  
SQL> INSERT INTO Seller VALUES  
2  (3, 'Fashion Hub', 'fashionhub@gmail.com', '9876543212', 'Mumbai');  
  
1 row created.  
  
SQL> INSERT INTO Seller VALUES  
2  (4, 'Tech Store', 'techstore@gmail.com', '9876543213', 'Hyderabad');  
  
1 row created.  
  
SQL> INSERT INTO Seller VALUES  
2  (5, 'Daily Needs', 'dailyneeds@gmail.com', '9876543214', 'Coimbatore');  
  
1 row created.  
  
SQL> INSERT INTO Seller VALUES  
2  (6, 'Watch House', 'watchhouse@gmail.com', '9876543215', 'Chennai');  
  
1 row created.  
  
SQL> INSERT INTO Seller VALUES  
2  (7, 'Home Store', 'homestore@gmail.com', '9876543216', 'Pune');  
  
1 row created.  
  
SQL> INSERT INTO Seller VALUES  
2  (8, 'Travel World', 'travelworld@gmail.com', '9876543217', 'Delhi');  
  
1 row created.
```

2.Create Inventory Table

The Inventory table maintains product stock details for each seller, including stock quantity, reorder level, and the last updated date.

Attribute	Data Type	Constraint
Inventory_ID	NUMBER	PRIMARY KEY
Seller_ID	NUMBER	FOREIGN KEY
Product_ID	NUMBER	FOREIGN KEY
Stock_Quantity	NUMBER	—
Reorder_Level	NUMBER	—
Last_Updated	DATE	—

```
SQL> CREATE TABLE Inventory (  
2     Inventory_ID NUMBER PRIMARY KEY,  
3     Seller_ID NUMBER,  
4     Product_ID NUMBER,  
5     Stock_Quantity NUMBER,  
6     Reorder_Level NUMBER,  
7     Last_Updated DATE,  
8     FOREIGN KEY (Seller_ID) REFERENCES Seller(Seller_ID),  
9     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)  
10 );
```

Table created.

```
SQL> INSERT INTO Inventory VALUES (1, 1, 23, 15, 5, SYSDATE);  
1 row created.  
  
SQL> INSERT INTO Inventory VALUES (2, 2, 24, 20, 5, SYSDATE);  
1 row created.  
  
SQL> INSERT INTO Inventory VALUES (3, 3, 25, 30, 10, SYSDATE);  
1 row created.  
  
SQL> INSERT INTO Inventory VALUES (4, 4, 26, 12, 5, SYSDATE);  
1 row created.  
  
SQL> INSERT INTO Inventory VALUES (5, 5, 27, 25, 8, SYSDATE);  
1 row created.  
  
SQL> INSERT INTO Inventory VALUES (6, 6, 28, 18, 5, SYSDATE);  
1 row created.  
  
SQL> INSERT INTO Inventory VALUES (7, 7, 29, 40, 10, SYSDATE);  
1 row created.  
  
SQL> INSERT INTO Inventory VALUES (8, 8, 30, 22, 5, SYSDATE);  
1 row created.
```

3..Establish Relationships Between Sellers, Products, and Stock

The Inventory table connects the Seller and Product tables using Seller_ID and Product_ID as foreign keys. This relationship helps maintain the stock information associated with each seller and product.

```

SQL> SELECT s.Seller_ID,
2      s.Seller_Name,
3      i.Inventory_ID,
4      i.Product_ID,
5      i.Stock_Quantity,
6      i.Reorder_Level
7 FROM Seller s
8 JOIN Inventory i
9 ON s.Seller_ID = i.Seller_ID;

```

SELLER_ID	SELLER_NAME	INVENTORY_ID
1	ABC Traders	1
23		
2	Smart World	2
24		
3	Fashion Hub	3
25		

PRODUCT_ID	STOCK_QUANTITY	REORDER_LEVEL
1	15	5
2	20	5
3	30	10

SELLER_ID	SELLER_NAME	INVENTORY_ID
4	Tech Store	4
26		
5	Daily Needs	5
27		
6	Watch House	6
28		

PRODUCT_ID	STOCK_QUANTITY	REORDER_LEVEL
4	12	5
5	25	8
6	18	5

SELLER_ID	SELLER_NAME	INVENTORY_ID
7	Home Store	7
29		

4.Maintain Seller Product Information

JOIN operation was used to combine the Seller, Inventory, and Product tables. It retrieves the seller name, product name, product ID, stock quantity, and reorder level in a single result.

JOIN Query:

```

SELECT s.Seller_ID,
s.Seller_Name,
p.Product_ID,
p.Product_Name,
i.Stock_Quantity,
i.Reorder_Level
FROM Seller s
JOIN Inventory i
ON s.Seller_ID = i.Seller_ID
JOIN Product p
ON i.Product_ID = p.Product_ID;

```

```

SQL> SELECT s.Seller_ID,
2         s.Seller_Name,
3         p.Product_ID,
4         p.Product_Name,
5         i.Stock_Quantity,
6         i.Reorder_Level
7 FROM Seller s
8 JOIN Inventory i
9 ON s.Seller_ID = i.Seller_ID
10 JOIN Product p
11 ON i.Product_ID = p.Product_ID;

SELLER_ID SELLER_NAME                                PRODUCT_ID
-----
PRODUCT_NAME
-----
STOCK_QUANTITY REORDER_LEVEL
-----
      1 ABC Traders                                23
Nike Running Shoes
      15              5
      2 Smart World                                24
Adidas Casual Shoes
      20              5

SELLER_ID SELLER_NAME                                PRODUCT_ID
-----
PRODUCT_NAME
-----
STOCK_QUANTITY REORDER_LEVEL
-----

```

5.Track Available and Unavailable Products

This operation is used to identify whether a product is available or unavailable based on its stock quantity. If the stock quantity is greater than 0, the product is considered Available; otherwise, it is considered Unavailable.

Input:

Product_ID, Stock_Quantity

```

SQL> SELECT Product_ID,
2         Stock_Quantity,
3         CASE
4             WHEN Stock_Quantity > 0 THEN 'Available'
5             ELSE 'Unavailable'
6         END AS Product_Status
7 FROM Inventory;

PRODUCT_ID STOCK_QUANTITY PRODUCT_STA
-----
      23          15 Available
      24          20 Available
      25          30 Available
      26          12 Available
      27          25 Available
      28          18 Available
      29          40 Available
      30          22 Available

8 rows selected.

```

```
SQL> SELECT Product_ID,
2         Stock_Quantity,
3         CASE
4             WHEN Stock_Quantity > 0 THEN 'Available'
5             ELSE 'Unavailable'
6         END AS Product_Status
7 FROM Inventory;
```

PRODUCT_ID	STOCK_QUANTITY	PRODUCT_STA
23	0	Unavailable
24	20	Available
25	30	Available
26	12	Available
27	25	Available
28	18	Available
29	40	Available
30	22	Available

3 rows selected.

Output:

Available / Unavailable

Generate Inventory Status Reports

This operation is used to generate an inventory status report by comparing the current stock quantity with the reorder level. It identifies whether the product is available, requires reordering, or is unavailable.

Input:

Product_ID, Stock_Quantity, Reorder_Level

```
SQL> SELECT Product_ID,
2         Stock_Quantity,
3         Reorder_Level,
4         CASE
5             WHEN Stock_Quantity = 0 THEN 'Unavailable'
6             WHEN Stock_Quantity <= Reorder_Level THEN 'Reorder Required'
7             ELSE 'Stock Available'
8         END AS Inventory_Status
9 FROM Inventory;
```

PRODUCT_ID	STOCK_QUANTITY	REORDER_LEVEL	INVENTORY_STATUS
23	0	5	Unavailable
24	20	5	Stock Available
25	30	10	Stock Available
26	12	5	Stock Available
27	25	8	Stock Available
28	18	5	Stock Available
29	40	10	Stock Available
30	22	5	Stock Available

3 rows selected.

Conclusion

The e-commerce database successfully manages, products, sellers, and inventory details using relational database concepts.

Primary keys, foreign keys, SQL operations, and JOIN queries are used to maintain relationships and retrieve meaningful information.

Inventory tracking and status reports help monitor product availability, stock levels, and reorder requirements efficiently.